

Today I Learned

A collection of references and demonstrations for concepts and methods that I have come across and want to mentally retain.

O957

Table of contents

Welcome!	3
How This Book Is Organized	3
Source	3
I Python	4
1 Check CSV Equivalence Using Polars	5
2 Find Empty PDFs Using Pathlib	7
3 Get The Remaining Dates In A Year Using Datetime	8
4 Uniformly Partitioning A String Using Textwrap	11
II R	12
5 Handling Unbound Tidyr Variables Using Rlang Data Masking	13
III VS Code / VSCodium	17
6 Opening VS Code From The Terminal Via Code	18
References	21

Welcome!

This site is the *Today I Learned* (TIL) collection of [O957](#).

Each entry:

- Records the date it was *learned*, the date it was *added*, and the last date on which it was substantially edited.
- Walks through the initial problem or material that prompted it, with runnable code where applicable.
- Links to the documentation or external resources consulted along the way.

How This Book Is Organized

Entries are grouped by programming language (e.g. Python) or discipline (e.g. Epidemiology).

New sections appear as entries accumulate. A list of resources referenced across all entries lives in the [References](#) chapter.

Source

The source for this book lives at <https://github.com/O957/O957-TIL> and is licensed under the [Apache License 2.0](#).

Corrections and suggestions are welcome via [issues](#), but please first consult the [contributing](#) and [code of conduct](#) files.

Part I

Python

1 Check CSV Equivalence Using Polars

Today I Learned added on 2025-11-20, learned on 2025-11-20; edited on 2026-06-14.

I found myself reviewing two PRs made by the same individual and each PR had a `csv` file containing forecasts. The `csvs` seemed identical, but I wasn't sure, so I wanted a way to check for equivalence using Python.

Given that, at present, I prefer `polars` over `pandas`, I searched `polars` for a method and found the method `equals` (see <https://docs.pola.rs/api/python/stable/reference/dataframe/api/polars.DataFrame.equals.html>) in the `DataFrame` class .

Example `csv` file as `example_01.csv`:

```
column_1,column_2
45,98
32,31
17,100
```

Example `csv` file as `example_02.csv`:

```
column_1,column_2
87,98
32,31
17,100
```

First, the non-Python, Unix command `diff` (the standard method I've employed for equivalence testing) can also be used.

```
diff example_01.csv example_02.csv
```

```
# output:
# 2c2
# < 45,98
# ---
# > 87,98
```

In Python, using the `equals` method of `polars`, which outputs `True` or `False`, instead of the actual differences if there are any:

```
import polars as pl

df_01 = pl.read_csv("example_01.csv")
df_02 = pl.read_csv("example_02.csv")
```

```
# output: False  
df_01.equals(df_02)
```

2 Find Empty PDFs Using Pathlib

Today I Learned added on 2025-11-20, learned on 2025-11-19; edited on 2026-06-14.

In generating routine forecast visualizations yesterday, I received an error thrown by `pypdf`, which indicated that the a PDF I was processing was empty (the forecast visualizations are aggregated into PDFs). After checking the usual things (i.e. that the internal R package was in fact up-to-date an installed and that I was logged-in and authenticated by Azure), I determined that I needed to actually find the empty PDF to resolve the error.

Since this seems common enough an issue, I expected there to be some one-line Python solutions. The solution I came up with may not be the best in terms of length or speed, but it worked for my use case:

```
import pathlib

pdf_files = [f for f in pathlib.Path(".").rglob("*.pdf") if
↳ f.stat().st_size == 0]
```

This line recursively gets all PDF files in the current `(".")` directory and all its sub-directories and adds them to the list if the size of the file is 0 (`if f.stat().st_size == 0`).

3 Get The Remaining Dates In A Year Using Datetime

Today I Learned added on 2025-11-13, learned on 2025-11-11; edited on 2026-06-14.

Suppose you want to see how many dates there are remaining in the current year or, better yet, want a `list` of all of the remaining dates in the current year.

The Python package `datetime`, which is Python's built-in date and time manipulation package (<https://docs.python.org/3/library/datetime.html>), allows one to do this rather easily.

```
import datetime

# out: datetime.date(2025, 11, 13)
today = datetime.date.today()

# out: datetime.date(2025, 12, 31)
end_of_year = today.replace(month=12, day=31)

# out: 49
remaining_days_in_year = (end_of_year - today).days + 1

# out: ['2025-11-13', '2025-11-14', ..., '2025-12-31']
remaining_dates_in_year = [today +
    ↪ datetime.timedelta(days=d).strftime("%Y-%m-%d") for d in
    ↪ range(remaining_days_in_year)]
```

I became interested in this originally when I wanted to create text files for my notes for the remainder of the year. I use files entitled `Notes_For_YYYY-DD-MM` containing a template (e.g. Todos, Daily Plan, etc...) and the task boiled down to getting the remaining [ISO-8601](#) formatted dates in the current year. The full program can be found below:

Notes File Generator

```
"""
Script for getting the remaining dates (formatted as
ISO-8601 strings) in the current year and creating
daily note files from a template.
"""

from datetime import date, timedelta
import os
import sys
```



```

def get_remaining_dates(year: int) -> list[str]:
    """
    Get all remaining dates in the specified year as
    ISO-8601 strings.

    Parameters
    -----
    year : int
        The year to get remaining dates for.

    Returns
    -----
    list[str]
        List of date strings in YYYY-MM-DD format.
    """
    today = date.today()
    end_of_year = date(year, 12, 31)
    days_remaining = (end_of_year - today).days + 1
    return [(today + timedelta(days=i)).strftime("%Y-%m-%d") for i in
        ↪ range(days_remaining)]

def read_template(filename: str) -> str:
    """
    Read the notes template file.

    Parameters
    -----
    filename : str
        Path to the template file.

    Returns
    -----
    str
        Template content.
    """
    try:
        with open(filename, "r") as f:
            return f.read()
    except FileNotFoundError:
        print(f"Error: Template file '{filename}' not found.")
        sys.exit(1)
    except IOError as e:
        print(f"Error reading template file: {e}")
        sys.exit(1)

def create_note_files(dates: list[str], template: str, prefix: str =
    ↪ "Notes_For_") -> int:
    """

```

Create note files for each date if they don't already exist.

Parameters

dates : list[str]

 List of date strings to create files for.

template : str

 Template content to write to each file.

prefix : str

 Filename prefix for note files.

Returns

int

 Number of files created.

"""

created_count = 0

for d in dates:

 filename = f"{prefix}{d}.txt"

 if not os.path.exists(filename):

 with open(filename, "w") as f:

 f.write(template)

 created_count += 1

return created_count

if __name__ == "__main__":

 current_year = date.today().year

 remaining_dates = get_remaining_dates(current_year)

 template = read_template("NOTES.txt")

 created = create_note_files(remaining_dates, template)

 print(f"Created {created} new note files for the remaining days of
 ↪ {current_year}.")

4 Uniformly Partitioning A String Using Textwrap

Today I Learned added on 2025-11-13, learned on 2025-11-08; edited on 2026-06-14.

A merchant on Ebay sent me a list of item ID codes (each 12 characters long) as a newline separated list. When I copied and pasted the list into a text file (to have two separate windows open at once), the characters were concatenated into a single string — how annoying! I tried different types of copying-and-pasting (pasting using **Cmd-V** and **Cmd-Shift-V**) to no avail. I then wondered if Python could help me avoid manually cutting out each 12 character string from the single long string. The answer is “Yes!” and there is a package called `textwrap` that can help with this task specifically.

```
# concatenated str
c_str = "432853242343242343242343242343276575"

# partition size
p_size = 12

# uniformly partitioned string; ['432853242343', '242343242342',
↪ '343243276575']
up_str = [c_str[i:i+p_size] for i in range(0, len(c_str), p_size) if
↪ len(c_str) % p_size == 0]
```

The simpler method is to use Python’s `textwrap` package:

```
import textwrap

c_str = "432853242343242343242343242343276575"

p_size = 12

if len(c_str) % p_size == 0:
    # ['432853242343', '242343242342', '343243276575']
    up_str = textwrap.wrap(c_str, p_size)
```

Part II

R

5 Handling Unbound Tidyr Variables Using Rlang Data Masking

Today I Learned added on 2025-11-18, learned on 2025-11-19; edited on 2026-06-14.

During some routine R-package development, I noticed a NOTE in a GitHub Actions (Run `r-lib/actions/check-r-package@v2`) failure, produced by the workflow `R-CMD-check.yaml`:

R CMD Check Workflow

```
# Workflow derived from https://github.com/r-lib/actions/tree/v2/examples
# Need help debugging build failures? Start at
# ↪ https://github.com/r-lib/actions#where-to-find-help
name: R-CMD-check.yaml

on:
  pull_request:
  push:
    branches: [main]

concurrency:
  group: ${{ github.workflow }}-${{ github.head_ref || github.ref }}
  cancel-in-progress: true

permissions: read-all

jobs:
  R-CMD-check:
    runs-on: ${{ matrix.config.os }}

    name: ${{ matrix.config.os }} (${{ matrix.config.r }})

    strategy:
      fail-fast: false
      matrix:
        config:
          - {os: macos-latest,  r: 'release'}
          - {os: windows-latest, r: 'release'}
          - {os: ubuntu-latest,  r: 'release'}

    env:
      GITHUB_PAT: ${{ secrets.GITHUB_TOKEN }}
      R_KEEP_PKG_SOURCE: yes

    steps:
```

```

- uses: actions/checkout@v4

- uses: r-lib/actions/setup-pandoc@v2

- uses: r-lib/actions/setup-r@v2
  with:
    r-version: ${ matrix.config.r }
    http-user-agent: ${ matrix.config.http-user-agent }
    use-public-rspm: true

- uses: r-lib/actions/setup-r-dependencies@v2
  with:
    extra-packages: any::rcmdcheck
    needs: check

- uses: r-lib/actions/check-r-package@v2
  with:
    upload-snapshots: true
    build_args: 'c("--no-manual", "--compact-vignettes=gs+qpdf")'

```

Here is a section of the NOTE:

```

* checking R code for possible problems ... NOTE
check_authorized_users: no visible binding for global variable '.data'
  (/Users/runner/work/hubhelpR/hubhelpR/check/hubhelpR.Rcheck/00_pkg_src/hubhelpR/R/check_au
47)
check_authorized_users: no visible global function definition for
  'na.omit'
  (/Users/runner/work/hubhelpR/hubhelpR/check/hubhelpR.Rcheck/00_pkg_src/hubhelpR/R/check_au
47)

```

For more context, here is the relevant code that the above segment refers to:

```

changed_dirs_tbl <- tibble::tibble(dir = changed_model_ids)

authorization_check <- changed_dirs_tbl |>
  dplyr::left_join(model_metadata, by = "dir", na_matches = "never") |>
  dplyr::group_by(.data$dir) |>
  dplyr::summarize(
    modifiable = all(tidyr::replace_na(.data$is_model_dir, FALSE)),
    actor_authorized = !!gh_actor %in% .data$designated_github_users,
    authorized_users = paste(
      na.omit(.data$designated_github_users),
      collapse = ", "
    ),
    has_authorized_users =
      ↪ length(na.omit(.data$designated_github_users)) > 0,
    .groups = "drop"
  )

```

The `tidyverse` documentation for `dplyr` has a section on this NOTE (see <https://dplyr.tidyverse.org/articles/in-packages.html>). In particular, they write:

If you're writing a package and you have a function that uses data masking or tidy selection:

```
my_summary_function <- function(data) {  
  data %>%  
    select(grp, x, y) %>%  
    filter(x > 0) %>%  
    group_by(grp) %>%  
    summarise(y = mean(y), n = n())  
}
```

You'll get an NOTE because R CMD check doesn't know that `dplyr` functions use tidy evaluation:

```
N checking R code for possible problems  
my_summary_function: no visible binding for global variable 'grp', 'x', 'y'  
Undefined global functions or variables:  
  grp x y
```

To eliminate this note:

- For data masking, import `.data` from `rlang` and then use `.data$var` instead of `var`.
- For tidy selection, use `"var"` instead of `var`.

That yields:

```
## @importFrom rlang .data  
my_summary_function <- function(data) {  
  data %>%  
    select("grp", "x", "y") %>%  
    filter(.data$x > 0) %>%  
    group_by(.data$grp) %>%  
    summarise(y = mean(.data$y), n = n())  
}
```

If I wanted to simply get rid of the NOTE, I could add `@importFrom rlang .data` to my problematic file and then import `rlang` in my DESCRIPTION file. Before exploring this further through a small example, I wanted to see what `rlang` had to say.

Both <https://rlang.r-lib.org/> and <https://rlang.r-lib.org/reference/topic-data-mask.html> provide copious and somewhat interesting reference information. These statements (which you can read about further) in the latter article, seemed more relevant:

To keep these objects and functions in scope, the data frame is inserted at the bottom of the current chain of environments. It comes first and has precedence over the user environment. In other words, it masks the user environment.

and

The tidyverse embraced the data-masking approach in packages like `ggplot2` and `dplyr` and eventually developed its own programming framework in the `rlang` package. None of this would have been possible without the following landmark developments from S and R authors.

For the example, I need the following:

- An `R-CMD-check.yaml` workflow file (see dropdown above).
- An R package using `dplyr`.

Steps:

- In R, run: `install.packages(c("devtools", "roxygen2"))`.

REMAINDER PENDING.

Part III

VS Code / VSCodium

6 Opening VS Code From The Terminal Via Code

Today I Learned added on 2025-11-30, learned on 2025-11-30; edited on 2026-06-14.

For my paid and unpaid programming work I've used [VS Code](#). The following has been routine for me for roughly 4 years: I open the target folder by pressing the **Open** button on VS Code's platform or, if I remember to, to **Cmd+O** on my MacOS to open a folder.

Today, I remarked internally to myself that VS Code probably has a method to open a new window from within the Terminal. Lo and behold, it does: `code .` to open the current directory in VS Code.

Of note, to get the `code` command working, I performed the following actions:

1. Open VS Code.
2. Open Command Palette via **Cmd+Shift+P**.
3. Type: "shell command"
4. Select: "Shell Command: Install 'code' command in PATH"
5. Gave permissions to VS Code.

Running `code --help` shows some other interesting options:

```
code --help
Visual Studio Code 1.105.1
```

```
Usage: code [options] [paths...]
```

To read from stdin, append '-' (e.g. 'ps aux | grep code | code -')

Options

<code>-d --diff <file> <file></code>	Compare two files with each other.
<code>-m --merge <path1> <path2> <base> <result></code>	Perform a three-way merge by providing paths for each file, the common origin of both modified versions, and the merge results.
<code>-a --add <folder></code>	Add folder(s) to the last active window.
<code>--remove <folder></code>	Remove folder(s) from the last active window.
<code>-g --goto <file:line[:character]></code>	Open a file at the path on the specified line and character.
<code>-n --new-window</code>	Force to open a new window.
<code>-r --reuse-window</code>	Force to open a file or folder in an already open window.
<code>-w --wait</code>	Wait for the files to be closed before returning.
<code>--locale <locale></code>	The locale to use (e.g. en-US or zh-TW).
<code>--user-data-dir <dir></code>	Specifies the directory that user data is kept for multiple distinct instances of Code.
<code>--profile <profileName></code>	Opens the provided folder or workspace with the provided profile.

the profile with the workspace. If the profile one is created.

`-h --help` Print usage.

Extensions Management

`--extensions-dir <dir>` Set the root path for extensions.

`--list-extensions` List the installed extensions.

`--show-versions` Show versions of installed extensions, when using `--list-extensions`.

`--category <category>` Filters installed extensions by provided category, when using `--list-extensions`.

`--install-extension <ext-id | path>` Installs or updates an extension. The argument is either a VSIX. The identifier of an extension is '`{publisher}.{name}@{version}`'. For example: '`vscode.csharp@1.2.3`'.
`force'` argument to update to latest version. To install a specific version, use '`@{version}`'. For example: '`vscode.csharp@1.2.3`'.

`--pre-release` Installs the pre-release version of the extension, when using `--install-extension`.

`--uninstall-extension <ext-id>` Uninstalls an extension.

`--update-extensions` Update the installed extensions.

`--enable-proposed-api <ext-id>` Enables proposed API features for extensions. Can receive multiple arguments to enable individually.

Model Context Protocol

`--add-mcp <json>` Adds a Model Context Protocol server definition to the user profile. Accepts a JSON object like `'{"name":"server-name","command":...}'`

Troubleshooting

`-v --version` Print version.

`--verbose` Print verbose output (implies `--wait`).

`--log <level>` Log level to use. Default is 'info'. Allowed values are 'warn', 'info', 'debug', 'trace', 'off'. You can add log level to the extension identifier by passing extension id and log level like '`{publisher}.{name}:{logLevel}`'. For example: '`ms-vscode.csharp:debug`'. You can receive one or more such entries.

`-s --status` Print process usage and diagnostics information.

`--prof-startup` Run CPU profiler during startup.

`--disable-extensions` Disable all installed extensions. This option is not available only when the command opens a new window.

`--disable-extension <ext-id>` Disable the provided extension. This option is not available only when the command opens a new window.

`--sync <on | off>` Turn sync on or off.

`--inspect-extensions <port>` Allow debugging and profiling of extensions. Check the developer tools for the connection URI.

`--inspect-brk-extensions <port>` Allow debugging and profiling of extensions with breakpoints after start. Check the developer tools for the connection URI.

`--disable-lcd-text` Disable LCD font rendering.

`--disable-gpu` Disable GPU hardware acceleration.

`--disable-chromium-sandbox` Use this option only when there is requirement to run the user on Linux or when running as an elevated user.

	Windows.
<code>--locate-shell-integration-path <shell></code>	Print the path to a terminal shell integration script for 'bash', 'pwsh', 'zsh' or 'fish'.
<code>--telemetry</code>	Shows all telemetry events which VS code collects
<code>--transient</code>	Run with temporary data and extension directories time.

Subcommands

<code>chat</code>	Pass in a prompt to run in a chat session in the current working directory.
<code>serve-web</code>	Run a server that displays the editor UI in browsers.
<code>tunnel</code>	Make the current machine accessible from <code>vscode.dev</code> or other machines through

References

Collected references across the TIL entries.